found by separating the data from one service to another within the database, sometimes using namespaces.

Our last decoupling feature is *time*. This feature is not new, of course. In any ESB, message brokers provide pub/sub communication to decouple the time of publication from the time of consumption. Queues or topics provide us with time decoupling by design and some leeway to control flow if the producer is faster than the consumer. The disadvantage of pub/sub is traceability. For the same reason that there is no direct contact between publisher and subscriber, it is harder to determine the impact of a message on all possible subscribers. That's why it's important with this architecture to get the observability and monitoring right.

The other way to decouple in time is—no surprise—an asynchronous peer-to-peer call. In this case, the client must provide a callback in the request that is executed when the response is sent or an error occurs. This feature is also related to isolating errors, because in a synchronous call, an error would propagate from the server to the client. A well-known example of these risks is a synchronous call over HTTP, which can be aborted while waiting for a response. If the server crashes, this raises an exception on the client that must be handled, and it waits in the meantime. Table B.1 shows the beneficial properties of isolation.

Table B.1 The benefits of isolation